Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL604**	**Course Name:**	**Machine Learning Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**01**
Title of the Experiment:	**To implement Data Preprocessing Techniques using NumPy, Pandas, and Matplotlib in Anaconda (Jupyter Notebook)/Google Colab/PyCharm Community Edition**
Date of Performance:	**08/01/2026**
Date of Submission:	**22/01/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Checked by

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Mrs Sneha Yadav

Signature :

Date:

Aim: To implement Data Preprocessing Techniques using NumPy, Pandas, and Matplotlib in Anaconda (Jupyter Notebook)/Google Colab/PyCharm Community Edition..

Objective: Ability to understand the different platforms and their function used for data preprocessing and model development.

Theory:

Anaconda
Anaconda is a popular platform that simplifies the process of installing, managing, and deploying data science tools and libraries. It is widely used for scientific computing, machine learning, and data analysis.

Key Features of Anaconda:
Package Management: Comes with conda, a package manager that helps install and manage Python packages and their dependencies.
Pre-installed Libraries: Includes popular data science libraries like NumPy, pandas, matplotlib, scikit-learn, and TensorFlow.
Jupyter Notebooks: Supports running Jupyter Notebook, an interactive environment for writing and running code, especially useful for data analysis and visualization.
Environment Management: Allows you to create isolated environments for different projects, ensuring there are no conflicts between dependencies.

Pros:
User-friendly and beginner-friendly.
Works offline after installation.
Robust environment management.

Cons:
The download and installation size is large.
May require manual updates for packages.

Google Colab
Google Colab (Collaboratory) is a free, cloud-based platform provided by Google that allows you to write and execute Python code in a Jupyter Notebook-like environment. It is especially useful for machine learning and data science tasks.

Key Features of Google Colab:
Cloud-Based: No installation required; everything runs in the browser.
GPU and TPU Support: Provides free access to GPUs and TPUs for accelerated computations. **Seamless Integration:** Easily integrates with Google Drive for saving and accessing notebooks.
Collaboration: Multiple users can work on the same notebook in real time.

Pros:
Free access to powerful computing resources (GPUs/TPUs).
Easy to share and collaborate.
Pre-installed popular libraries like TensorFlow, Keras, and pandas.

Cons:
Requires an internet connection.
Limited runtime (usually disconnects after a few hours of inactivity).

Core Features of PyCharm Community Edition
Python Support
Full support for Python syntax
Code highlighting, auto-indentation, and code formatting
Smart Code Assistance
Code completion (basic & smart)
Syntax & error detection
Quick fixes and suggestions
Code navigation (go to definition, find usages)
Debugging & Testing
Powerful Python debugger
Breakpoints, step-into, step-over, variable inspection
Project & File Management
Easy project
creation Built-in file
explorer
Supports virtual environments (venv)
Cross-Platform
Works on Windows, macOS, and Linux

Feature	Anaconda	PyCharm (Community Edition)	Google Colab
Type	Python distribution	IDE (Integrated Development Environment)	Cloud-based Notebook
Platform	Windows, Linux, macOS	Windows, Linux, macOS	Web browser
Installation Required	Yes	Yes	No
Internet Required	No (after install)	No (after install)	Yes
Primary Use	Data Science & ML	Python software development	ML, DL, Research

Python Support	Excellent	Excellent	Excellent
Package Management	Conda, pip	pip, venv	pip (limited control)

Feature	Anaconda	PyCharm (Community Edition)	Google Colab
Pre-installed Libraries	NumPy, Pandas, SciPy, Matplotlib, TensorFlow	None (user installs)	Many ML libraries pre-installed
IDE Support	Spyder, Jupyter	Full-featured IDE	Notebook interface only
Code Editor	Basic (Spyder)	Advanced (smart editor)	Basic notebook cells
Debugger	Limited	Powerful debugger	Very limited
Version Control (Git)	Manual	Built-in Git support	Manual
Jupyter Notebook	Yes	No (Community Edition)	Yes (native)
Machine Learning Support	Very good	Manual setup required	Excellent
Deep Learning (GPU/TPU)	CPU only (by default)	CPU only	Free GPU & TPU
Collaboration	No	No	Yes (real-time sharing)
Cloud Storage	No	No	Google Drive
Offline Usage	Yes	Yes	No
Performance	Heavy (large size)	Fast & efficient	Depends on internet
Customization	Medium	High	Low
Cost	Free	Free	Free
Best For	Data science learners	Software developers	Researchers & students

Data preprocessing is the process of cleaning, transforming, and organizing raw data into a suitable format so that it can be effectively used for data analysis and machine learning models.

o Key Steps in Data Preprocessing

- Data Cleaning – Handling missing values, removing noise and duplicates
- Data Transformation – Normalization, scaling, encoding categorical data

- Data Reduction – Feature selection and dimensionality reduction
- Data Integration – Combining data from multiple sources

Purpose

- Improves data quality
- Increases model accuracy
- Reduces errors and bias
- Makes data machine-readable

Implementation:

1. **Import Required Libraries**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2. **Create a Sample Dataset**

```
data = {
    'Age': [22, 25, np.nan, 35, 40, 25],
    'Salary': [25000, 30000, 28000, np.nan, 45000, 30000],
    'Experience': [1, 3, 2, 5, np.nan, 3]
}

df = pd.DataFrame(data)
df
```

	Age	Salary	Experience
0	22.0	25000.0	1.0
1	25.0	30000.0	3.0
2	NaN	28000.0	2.0
3	35.0	NaN	5.0
4	40.0	45000.0	NaN
5	25.0	30000.0	3.0

3. **Identify Missing Values**

```
df.isnull()
df.isnull().sum()
```

```
Age           1
Salary        1
Experience    1
dtype: int64
```

4. Handling Missing Values

```
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Experience'] = df['Experience'].fillna(df['Experience'].mean())
df
```

	Age	Salary	Experience
0	22.0	25000.0	1.0
1	25.0	30000.0	3.0
2	29.4	28000.0	2.0
3	35.0	31600.0	5.0
4	40.0	45000.0	2.8
5	25.0	30000.0	3.0

5. Removing Duplicate Data

```
df.duplicated()
df.drop_duplicates(inplace=True)
df
```

	Age	Salary	Experience
0	22.0	25000.0	1.0
1	25.0	30000.0	3.0
2	29.4	28000.0	2.0
3	35.0	31600.0	5.0
4	40.0	45000.0	2.8

6. Data Normalization (Feature Scaling)

Min–Max Normalization

Values scaled between 0 and 1

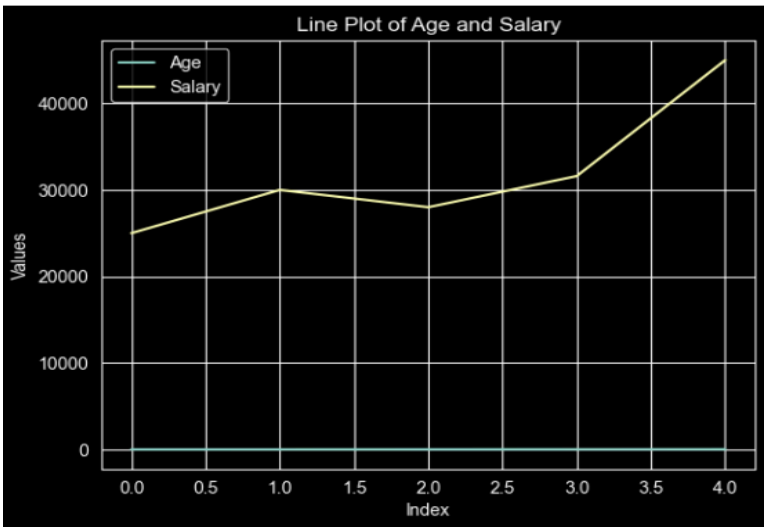
```
df_normalized = (df - df.min()) / (df.max() - df.min())
df_normalized
```

	Age	Salary	Experience
0	0.000000	0.00	0.00
1	0.166667	0.25	0.50
2	0.411111	0.15	0.25
3	0.722222	0.33	1.00
4	1.000000	1.00	0.45

7. Data Visualization using Matplotlib

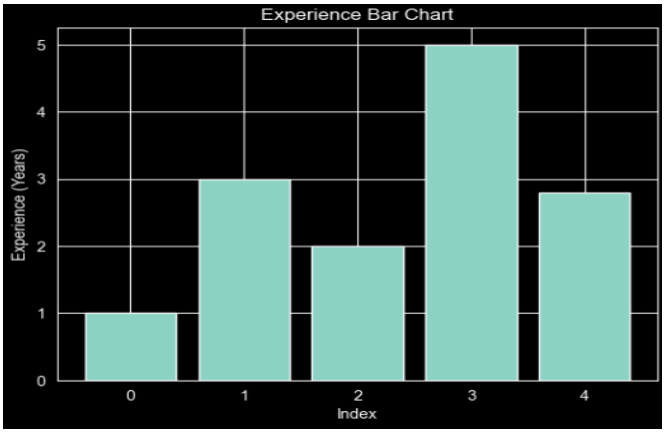
1. Line Plot

```
plt.plot(df['Age'], label='Age')
plt.plot(df['Salary'], label='Salary')
plt.legend()
plt.title("Line Plot of Age and Salary")
plt.xlabel("Index")
plt.ylabel("Values")
plt.show()
```



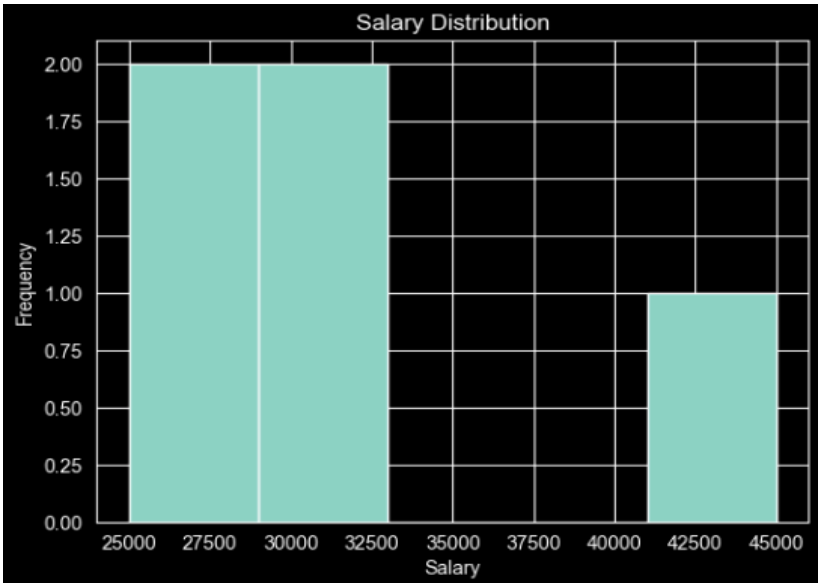
2. Bar Chart

```
plt.bar(df.index, df['Experience'])
plt.title("Experience Bar Chart")
plt.xlabel("Index")
plt.ylabel("Experience (Years)")
plt.show()
```



3. Histogram

```
plt.hist(df['Salary'], bins=5)
plt.title("Salary Distribution")
plt.xlabel("Salary")
plt.ylabel("Frequency")
plt.show()
```



Conclusion:

Pandas acts as the "manager," handling the data structure and cleaning operations like filling missing values and removing duplicates.

NumPy serves as the "mathematician," providing the logic for numerical operations and handling null value (`NaN`) definitions.

Matplotlib is the "visualizer," allowing you to verify that your cleaning and scaling (like Min-Max Normalization) actually worked through charts and graph